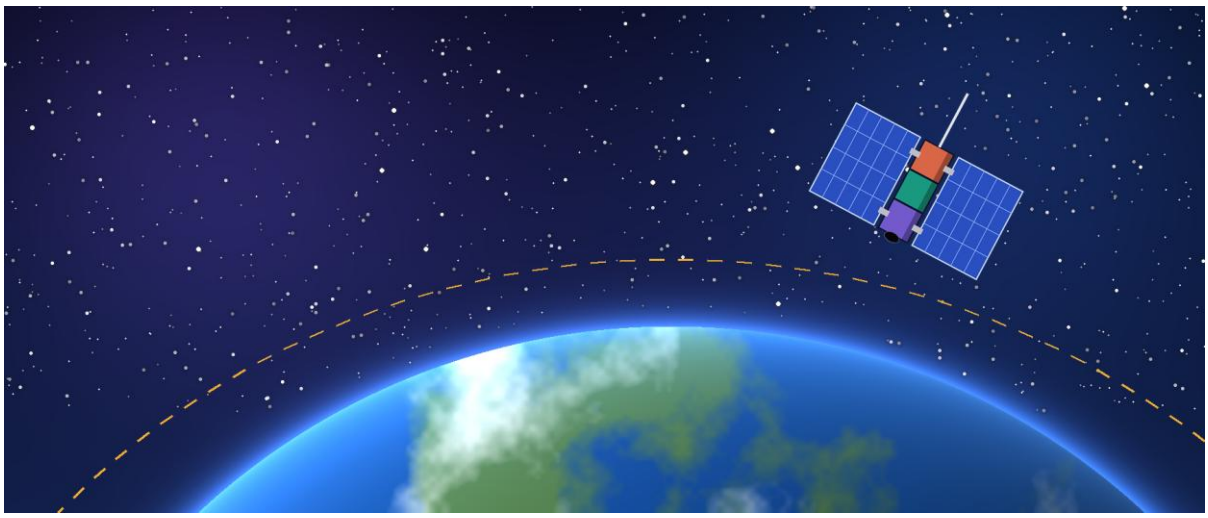


A Project Report

AeroSight

Satellite Weather Detection and Orbital Object Tracking

A software-only 3U CubeSat digital twin built with open-source tools

---

Project type	Learning and simulation project
Domain	Space systems, artificial intelligence, embedded electronics
Platform	Windows 11 with WSL2 Ubuntu, on a single laptop
Key tools	Python, YOLOv8, FreeCAD, KiCad, Verilog, Yosys, Blender
Date	October 2026

"Dreams don't need a lab to start."

Learned it. Built it. Did it.

Contents

Abstract 1
1. Introduction 2
2. System Overview 3
3. Methodology and Working Flow 4
4. Subsystem Design and Implementation 5
5. Results and Discussion 14
6. Challenges and Lessons Learned 14
7. Limitations 15
8. Future Work 15
9. How to Rebuild This Project 15
10. Conclusion 16
Tools and Credits 17

Abstract

AeroSight is a complete, software-only digital twin of a 3U CubeSat. It was designed, simulated and visualized on a single laptop using only free and open-source tools. The project joins an artificial-intelligence payload (weather classification and object detection), flight software, a database and a live dashboard with an orbit simulation, a modeled satellite body, a power-supply circuit board, a UHF antenna analysis, a small digital chip and a rendered three-dimensional overview. Each subsystem was built step by step, checked with the verification features of its own tool, and linked to the others. This report records the design, the working flow, the results and the lessons learned, so that the project can be repeated and extended by any student with a laptop.

1. Introduction

1.1 Background

A CubeSat is a small, standardized satellite built from 10 cm cubic units, each called 1U. A 3U CubeSat measures 10 × 10 × 30 cm and is the most common size for student, start-up and research missions. A real satellite needs a clean room, environmental testing, radio licensing and a launch, but almost every engineering decision behind it can first be studied in software. A virtual model that behaves like the real system is called a *digital twin*, and this project is built on that idea.

1.2 Motivation

The project began with a long-held ambition: to build a satellite from scratch. With no laboratory and no hardware budget, the practical route was to build the whole satellite virtually and to learn each discipline by doing it: orbital mechanics, mechanical design, electronics, antennas, digital logic, software and artificial intelligence.

1.3 Objectives

- Design every major subsystem of a 3U CubeSat using only open-source software.
- Detect weather conditions and objects in images with AI, and store each result in a database.
- Simulate a 500 km sun-synchronous orbit and display its ground track on a world map.
- Model the satellite body, design a power board, analyze a UHF antenna and design a small digital chip.
- Link all parts into one workflow and present them as a rendered and animated overview.

1.4 Scope and constraints

Everything runs on one laptop (Windows 11 with an Ubuntu environment under WSL2). No physical hardware is built, so all results are simulated. The weather module in this first version uses transparent colour rules rather than a trained network, which keeps it easy to understand and leaves clear room for improvement.

2. System Overview

2.1 Mission definition

Table 1. Mission summary.

Platform	3U CubeSat, 10 × 10 × 30 cm
Orbit	500 km sun-synchronous, inclination 97.4°
Payload	Camera with AI weather and object detection
Communication	437 MHz UHF half-wave dipole antenna
Power	Solar input, Li-ion battery, 3.3 V regulator
Onboard logic	Beacon timer chip written in Verilog
Development	Windows 11 and WSL2 Ubuntu, open-source tools
Status	Simulated digital twin (not launched)

2.2 Architecture

The system has three layers. The **software chain** carries a photo through AI detection and the flight software into a database and a dashboard. The **hardware models** (structure, power board, chip and antenna) describe the physical satellite and feed the final three-dimensional overview. The **orbit** layer calculates where the satellite flies and draws its path. The computer chip is shown beneath the flight software because it is the processor that would run that software.

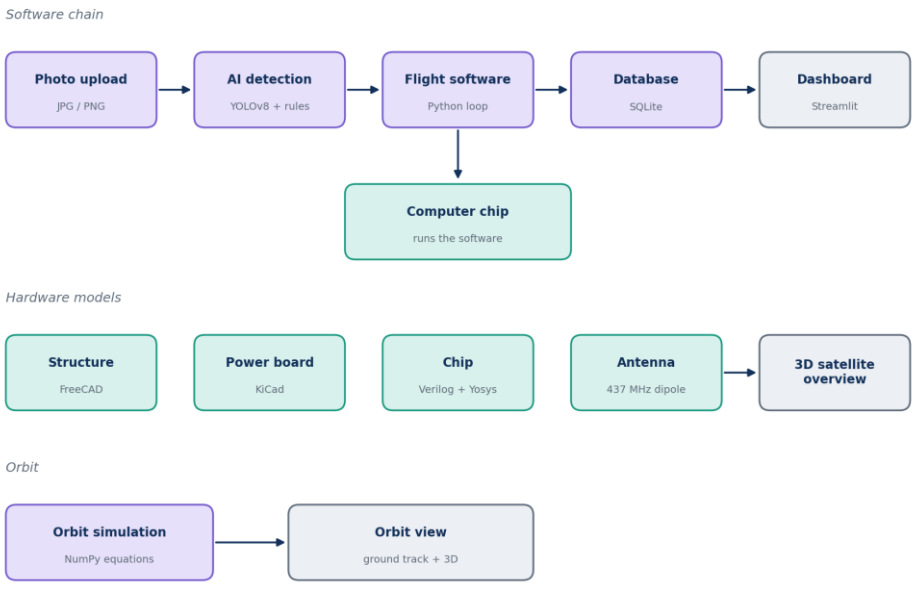


Figure 1. Architecture of the AeroSight project in three layers.

2.3 Tools used

Table 2. Tools, purpose and output of each subsystem.

Subsystem	Tool	Purpose	Output
AI payload	Python, OpenCV, YOLOv8 (Ultralytics, PyTorch)	Weather rules, object detection	Label and confidence
Flight software	Python	Main processing loop	detect.py
Database	SQLite	Store every result	aerosight.db
Dashboard	Streamlit, Pandas	Live results page	Web page
Orbit	Python (NumPy, Pandas)	Orbit equations, ground track	orbit.csv
Structure	FreeCAD	3U body model	.FCStd, .stl
Power board	KiCad	Schematic and PCB	.kicad_sch, .kicad_pcb
Antenna	Python (NumPy, Matplotlib)	Dipole pattern	PNG plot
Chip	Verilog, Icarus Verilog, Verilator, GTKWave, Yosys	Design, simulate, synthesize	Waveform, netlist
Overview	Blender, Three.js	Render and animation	PNG, web page

3. Methodology and Working Flow

The project was built in eight steps, each one producing a file that the next step could use. The order matters: the AI payload and software come first because they give working results early, and the visual steps come last because they depend on every earlier part.

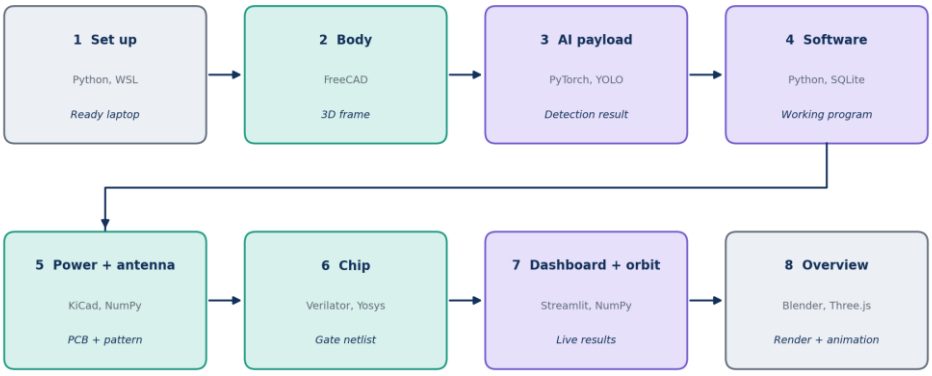


Figure 2. The eight-step workflow from laptop setup to the final overview.

At run time the data flow is simple. A photo is placed in the `photos` folder. The flight software reads it, estimates the weather, runs the object detector, writes one row to the database and saves a copy of the image with boxes drawn on it. The dashboard reads the database and the

saved images and shows them in a browser. Separately, the orbit script writes a file of latitude and longitude points that the dashboard draws on a map.

> *Design principle: build the smallest working version of each part first, verify it with the tool's own checker, and only then connect it to the next part.*

4. Subsystem Design and Implementation

4.1 AI payload: weather and object detection

The payload analyzes each photo in two ways. **Weather** is estimated by converting the image to the HSV colour space and measuring what share of pixels are dark, white or blue. **Objects** are found with YOLOv8n, a small pretrained detector that recognizes the 80 classes of the COCO dataset, including cars, people, buses and aircraft.

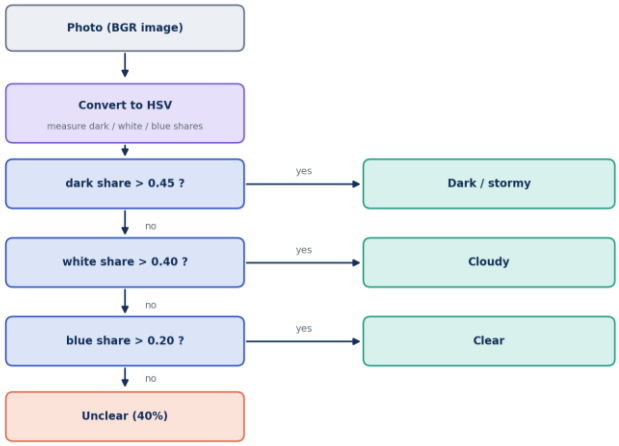


Figure 3. Decision rules used by the first-version weather classifier.

The confidence value is 0.5 plus the share that triggered the rule, limited to 95 percent. When no rule fires, the result is "unclear" at a fixed 40 percent, which is an honest way of saying that the program does not know.

```
def classify_weather(img):
    h, s, v = cv2.split(cv2.cvtColor(img, cv2.COLOR_BGR2HSV))
    blue  = np.sum((h>=90)&(h<=130)&(s>60)&(v>100)) / h.size
    white = np.sum((s<40) & (v>170)) / h.size
    dark  = np.sum(v<80) / h.size
    if dark  > 0.45: return "dark/stormy", min(0.5+dark, 0.95)
    if white > 0.40: return "cloudy",      min(0.5+white,0.95)
    if blue  > 0.20: return "clear",       min(0.5+blue, 0.95)
    return "unclear", 0.40
```

Table 3. Detection results for the three test photos.

Photo	Weather (confidence)	Objects found
CarImage	clear (71%)	car, person
SkyImage	cloudy (95%)	bus, car
StreetLightImage	unclear (40%)	none

The object detections are reasonable, with boxes on the cars and people in the street scene. The weather rules are the weak part: the sunset sky in the first photo was called “clear” because it is blue and orange rather than grey, and the cloudy street-light photo was called “unclear”. This is expected from colour rules, and it is the main reason a trained classifier is the first item in the future work.

4.2 Flight software, database and dashboard

The flight software is a single Python loop. For every image it classifies the weather, detects objects, inserts one record into an SQLite database and saves the annotated image to the `outputs` folder. SQLite was chosen because it is a single file, needs no server and is the same kind of store a small onboard computer could use.

Table 4. Structure of the detections table.

Column	Type	Meaning
id	INTEGER	Automatic record number
time	TEXT	When the photo was analyzed
photo	TEXT	File name of the image
weather	TEXT	dark/stormy, cloudy, clear or unclear
weather_conf	REAL	Confidence between 0 and 1
objects	TEXT	Detected object names, separated by commas

The dashboard is a Streamlit page started with one command. It shows the number of photos analyzed, the results table, a bar chart of weather types, the annotated photos and the orbit map.



Figure 4. Dashboard: metrics and the detections table read from the database.

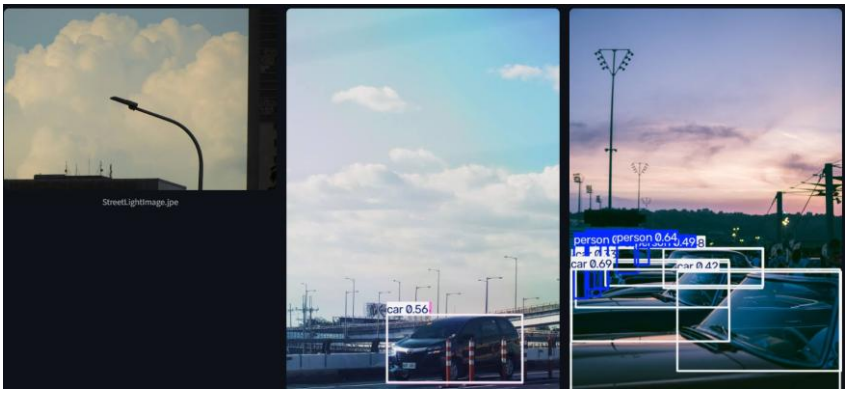


Figure 5. Dashboard: annotated photos with detected cars and people.

4.3 Orbit simulation

A circular orbit at 500 km altitude was chosen because it is typical for small Earth-observation satellites. Its inclination of 97.4° makes it sun-synchronous: the satellite crosses the same latitude at the same local solar time on every pass, which gives similar lighting in every image. The motion follows from Newton's law of gravitation:

$$a = R_E + h = 6378.137 + 500 = 6878.137\ km$$

$$n = \sqrt{(\mu / a^3)} = 1.1068 \times 10^{-3}\ rad/s, \quad \mu = 398\,600.44\ km^3/s^2$$

$$T = 2\pi / n \approx 5677\ s \approx 94.6\ min, \quad v = \sqrt{(\mu / a)} \approx 7.61\ km/s$$

The satellite position is rotated into an Earth-fixed frame using the Earth's rotation rate of 7.292 × 10^{-5} rad/s, and then converted to latitude and longitude every 30 seconds.

Table 5. Orbit parameters calculated by the simulation.

Parameter	Value
Altitude	500 km
Inclination	97.4° (sun-synchronous)
Orbital period	94.6 minutes
Orbital speed	7.61 km/s
Orbits per day	15.2
Westward shift per orbit	about 23.7° of longitude

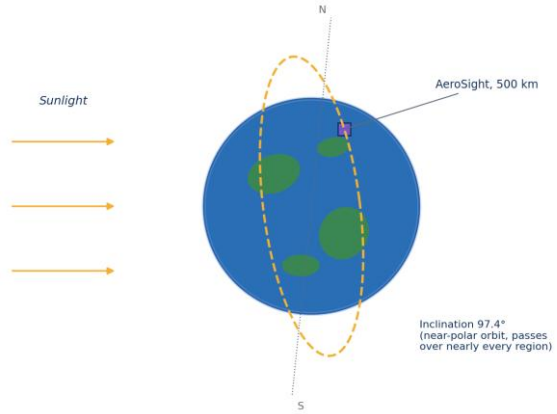


Figure 6. A near-polar, sun-synchronous orbit seen from the side.



Figure 7. Simulated ground track of three orbits, drawn on the dashboard.

On the map, each curve is one lap. Every new lap lies about 23.7° to the west of the previous one because the Earth turns beneath the orbit. Over a day, the 15 laps spread across the whole globe, which is why this orbit suits weather and Earth imaging.

4.4 Satellite structure

The body was modeled in FreeCAD by a short Python macro, which makes the design repeatable: change a number and run it again. It contains three 1U sections, a lens, two hinged solar panels and an antenna rod. The model was exported as an STL file and its dimensions were reused in the Blender scene.

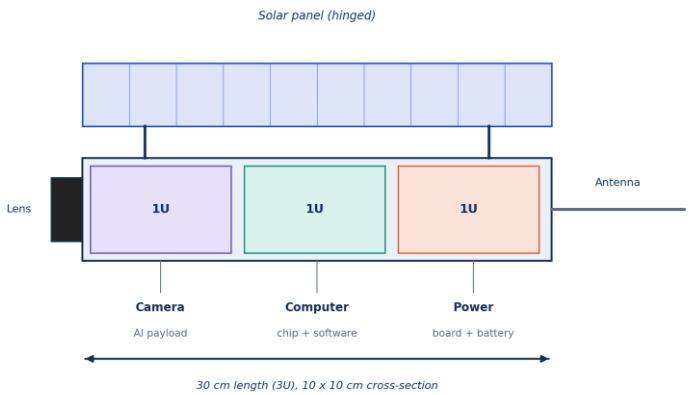


Figure 8. Cut-away view of the 3U body and its parts.

Table 6. Parts of the structural model.

Part	Size (mm)	Role
Camera section	100 × 100 × 100	Imaging payload with the lens facing Earth
Computer section	100 × 100 × 100	Onboard computer and chip
Power section	100 × 100 × 100	Power board and battery
Solar panels (2)	200 × 3 × 300 each	Generate power; hinged to the body
Antenna rod	170 long, 3 diameter	Visual stand-in for the deployed dipole

> *The 170 mm rod is only a visual marker. The real 437 MHz dipole is 32.6 cm long, made of two 16.3 cm arms, as calculated in Section 4.6.*

4.5 Power board

The power board turns raw solar input into steady power for the computer. Current passes through a Schottky diode, which blocks reverse flow, into a charge controller that fills a Li-ion battery. A linear regulator then produces 3.3 V for the electronics. The charge current is set by one resistor: I = 1000 / R(kΩ) mA, so the 2 kΩ resistor gives 500 mA.

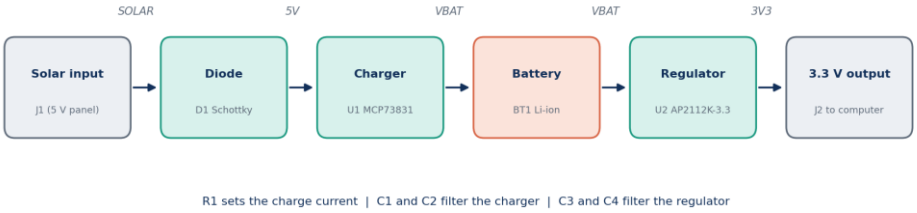


Figure 9. Power chain from the solar input to the 3.3 V output.

Table 7. Components of the power board (11 parts).

Ref	Part	Value / footprint	Role
J1	Connector 1×2	Pin header 2.54 mm	Solar panel input
D1	Schottky diode	SOD-123	Blocks reverse current
U1	MCP73831-2-OT	SOT-23-5	Li-ion charge controller
R1	Resistor	2 kΩ, 0805	Sets 500 mA charge current
C1, C2	Capacitors	4.7 µF, 0805	Charger input and battery filters
BT1	Battery connector	Pin header 2.54 mm	Li-ion cell, connected by wires
U2	AP2112K-3.3	SOT-23-5	3.3 V regulator
C3, C4	Capacitors	1 µF, 0805	Regulator input and output filters
J2	Connector 1×2	Pin header 2.54 mm	3.3 V output to the computer

Table 8. Connections (nets) of the power board. U1 pin 1 is unused.

Net	Connected pins
SOLAR	J1.1, D1.2
5V	D1.1, U1.4 (VDD), C1.1
PROG	U1.5, R1.1
VBAT	U1.3, BT1.1, C2.1, U2.1 (VIN), U2.3 (EN), C3.1
3V3	U2.5 (VOUT), C4.1, J2.1
GND	J1.2, J2.2, BT1.2, C1.2, C2.2, C3.2, C4.2, R1.2, U1.2, U2.2

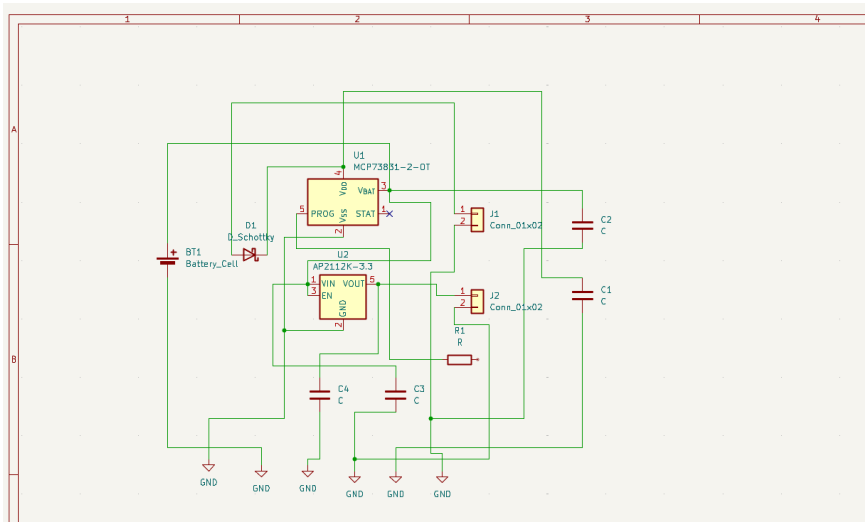


Figure 10. KiCad schematic of the power board.

The schematic passed the electrical rules check with no errors after power flags were added to the 5 V and ground nets. The board was then laid out and routed. The design rules check finished with **0 errors, 0 warnings and 0 unconnected items**.

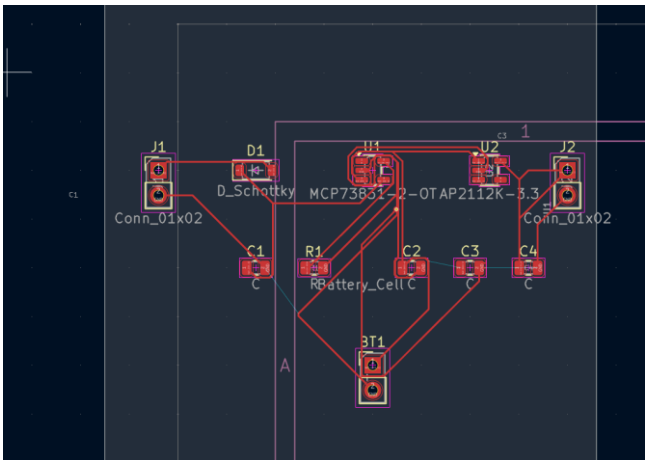


Figure 11. PCB layout of the power board, with all connections routed.

4.6 UHF antenna

A half-wave dipole is the simplest reliable antenna for a small satellite. At 437 MHz, a common amateur-satellite frequency, its length follows from the wavelength. The arms are shortened by 5 percent to account for real wire.

$$\lambda = c / f = 299\,792\,458 / 437 \times 10^{6} = 0.686\ m$$

$$arm\ length = 0.95 \times \lambda / 4 = 0.163\ m, \quad total\ length = 32.6\ cm$$

Table 9. Antenna design values.

Property	Value
Frequency	437 MHz
Wavelength	68.6 cm
Total length	32.6 cm (2 × 16.3 cm)
Peak gain	2.15 dBi (ideal half-wave dipole)
Half-power beamwidth	about 78°
Pattern	Doughnut shape, no signal along the antenna axis

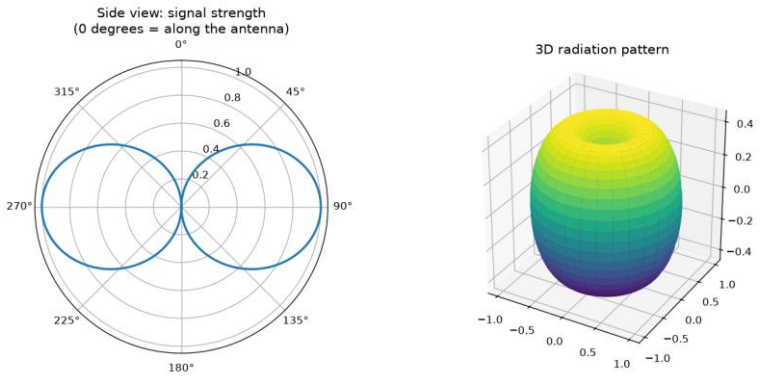


Figure 12. Radiation pattern: side view (left) and three-dimensional shape (right).

The doughnut pattern means the antenna radiates evenly around its axis. For a satellite that may tumble, this is useful, because there is no single pointing direction to maintain. The plot is the ideal theoretical pattern; a full-wave simulation of the real body would show some distortion from the metal structure.

4.7 Digital chip: beacon timer

Satellites send a regular “beacon” to show they are alive. The chip designed here does exactly that: a 16-bit timer counts clock cycles, and every eighth cycle it raises a one-cycle `beacon_tx` pulse while an 8-bit counter records how many beacons were sent.

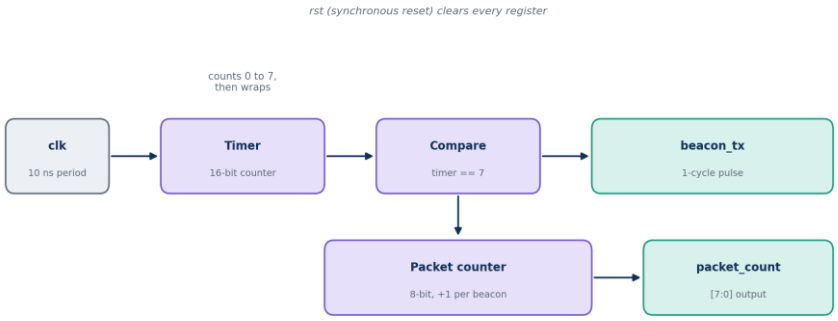


Figure 13. Block diagram of the beacon timer chip.

```
module beacon #(parameter PERIOD = 8) (
    input  wire clk, input wire rst,
    output reg beacon_tx, output reg [7:0] packet_count);
  reg [15:0] timer;
  always @(posedge clk) begin
    if (rst) begin timer<=0; beacon_tx<=0; packet_count<=0; end
    else begin
      beacon_tx <= 0;
      if (timer == PERIOD-1) begin
        timer<=0; beacon_tx<=1; packet_count<=packet_count+1;
      end else timer <= timer + 1;
    end
  end
endmodule
```

A test bench drove the chip with a 10 ns clock. The waveform shows the timer counting from 0 to 7, a beacon pulse every 80 ns, and the packet counter stepping up with each pulse.

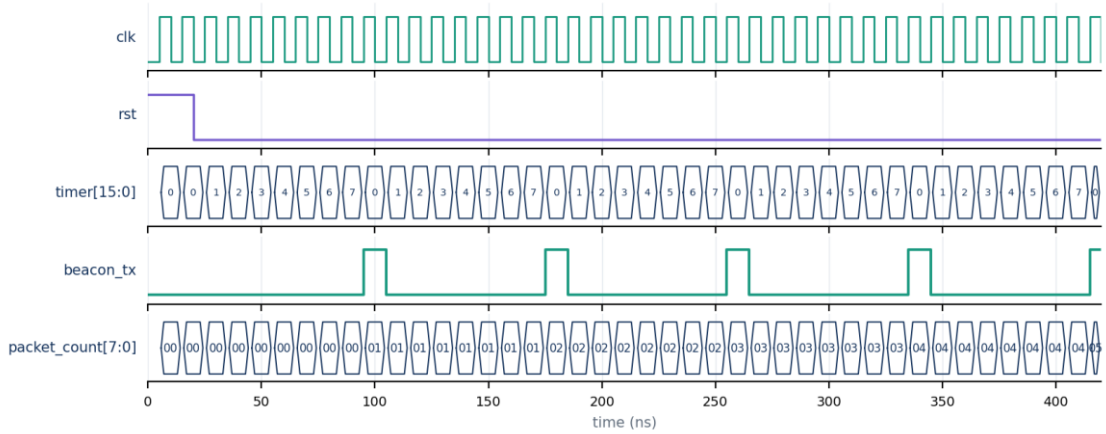


Figure 14. Simulation waveform: clock, reset, timer, beacon pulse and packet counter.

Yosys then converted the Verilog into basic logic gates. The result is 94 cells, of which 25 are flip-flops, matching the design: 16 for the timer, 8 for the packet counter and 1 for the beacon output.

Table 10. Gate-level result from Yosys synthesis.

Cell type (Yosys name)	Count	Role
$_SDFF_PP0_	17	Timer bits and beacon output (flip-flops)
$_SDFFE_PP0P_	8	Packet counter bits (flip-flops with enable)
$_OR_ / $_AND_ / $_ANDNOT_	19 / 4 / 13	Counting and compare logic
$_XOR_ / $_XNOR_	14 / 8	Adders for the two counters
$_NAND_ / $_NOT_	9 / 2	Remaining logic
Total	**94**	25 flip-flops and 69 logic gates

4.8 Visualization

The final overview was built in Blender with a Python script that recreates the satellite from the FreeCAD dimensions, adds a sun light and a fill light, places a large Earth in the background and renders the scene. A second, interactive version runs in a web browser: Earth turns, the satellite flies its orbit, and a follow mode shows it up close.

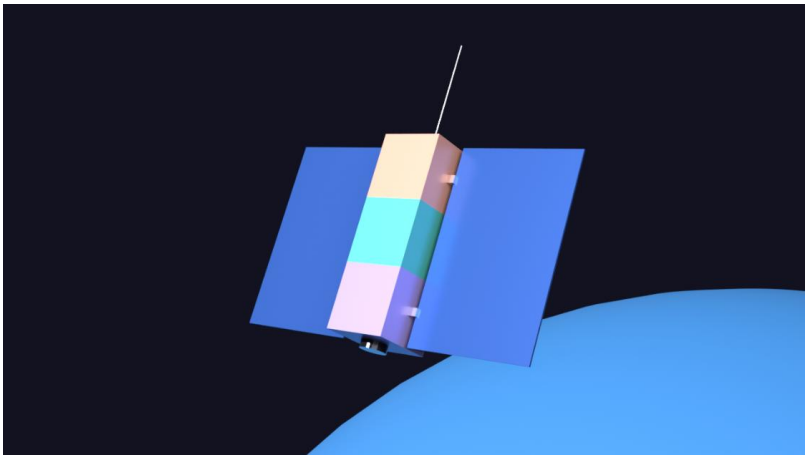


Figure 15. Blender render of the AeroSight satellite above the Earth.

5. Results and Discussion

Table 11. Summary of verified results.

Subsystem	Verified result
AI payload	Three photos analyzed; objects found in two; weather labels produced for all three
Database	Records saved to aerosight.db with time, photo, weather, confidence and objects
Dashboard	Live page showing metrics, table, chart, annotated photos and orbit map
Orbit	Period 94.6 min, 15.2 orbits per day, ground track plotted
Structure	3U model with seven parts, 100 × 100 × 300 mm
Power board	11 parts, 28 pads, 8 nets; 0 errors, 0 warnings, 0 unconnected items
Antenna	32.6 cm dipole at 437 MHz, 2.15 dBi, 78° beamwidth
Chip	Beacon pulse every 8 cycles in simulation; 94 cells after synthesis
Overview	Blender render at 1280 × 720 and an interactive animated view

Taken together, the results show that a full satellite workflow can be learned and demonstrated without hardware. The objective checks (rules check, design-rules check, simulation and synthesis) give real evidence that each part is internally consistent, even though none of it has flown.

6. Challenges and Lessons Learned

Most of the learning came from errors. The table lists the problems met during the build and how each was solved. Each fix became a rule for future projects.

Table 12. Problems met during the build and how they were solved.

Problem	Cause	Fix
“No photos found”	Image files ended in .jpe, which the script did not accept	Added more file extensions to the script
“No module named torch”	Large install was interrupted	Installed PyTorch first, then the other libraries
Ubuntu “package not found”	Package list had never been updated	Ran sudo apt update, then installed again
All parts got a capacitor footprint	Several rows were selected during assignment	Assigned footprints one group at a time
“Power pin not driven” errors	No power source marked on the nets	Added power flags to 5 V and ground
Unconnected items and a stray via	Tracks stopped short of pads	Extended the tracks, deleted the via, filled the ground plane
Dark, flat first render	One light and a side-on camera	Added a fill light, tilted the satellite, enlarged the Earth

> *Lesson: read the exact error message, find the one thing it names, and fix only that. Large systems become manageable when each part is checked on its own before it is connected.*

7. Limitations

- The satellite is a simulated digital twin; no hardware was built, tested for radiation or launched.
- The weather module uses colour rules, and it is only a rough guide. Ordinary photos are not real satellite images.
- The orbit is an ideal circular orbit and does not include atmospheric drag or the Earth's non-spherical shape.
- The antenna pattern is theoretical; a full-wave simulation of the real body was not run.
- The chip is verified by simulation and synthesis only; no physical layout was produced.
- Commercial chips are not radiation-hardened, so the beacon chip would be an experimental payload, not the flight computer.

8. Future Work

- Train a weather classifier on real satellite imagery from public archives such as NASA Worldview and Sentinel Hub.
- Replace the ideal orbit with real two-line element data for live tracking, and add object tracking over time.
- Take the beacon chip through the OpenLane flow to a physical layout and view it in KLayout.
- Run a full-wave antenna simulation with openEMS on the complete satellite body.
- Add a RISC-V processor core and run the AI model on edge hardware.
- Use F Prime or cFS for flight software, and SatNOGS for a ground-station link.

9. How to Rebuild This Project

This section is a short recipe. Anyone with a Windows laptop can follow it in order.

9.1 Install the tools

```
winget install Python.Python.3.12
winget install Microsoft.VisualStudioCode
winget install FreeCAD.FreeCAD
winget install KiCad.KiCad
winget install BlenderFoundation.Blender

cd %USERPROFILE%\Desktop && mkdir AeroSight && cd AeroSight
python -m venv venv && venv\Scripts\activate
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install opencv-python ultralytics pandas streamlit matplotlib

# inside Ubuntu (WSL)
sudo apt update
sudo apt install -y verilator yosys iverilog gtkwave klayout
```

9.2 Project folder

```
AeroSight/
  photos/          input images
  outputs/         annotated images
  chip/            beacon.v, tb.v
  detect.py        AI payload and database
  dashboard.py     Streamlit dashboard
  orbit.py         orbit simulation
  antenna.py       antenna pattern
  freecad_body.py  satellite body macro
```

9.3 Build order

1. Run `python detect.py` with two or three photos in the photos folder.
2. Start the dashboard with `streamlit run dashboard.py` and open localhost:8501.
3. Run `python orbit.py`, then refresh the dashboard to see the ground track.
4. Run the FreeCAD macro to build the body, then save and export it.
5. Draw the power board in KiCad: schematic, footprints, rules check, PCB, design-rules check.
6. Run `python antenna.py` to plot the antenna pattern.
7. In Ubuntu, run `iverilog -o sim beacon.v tb.v`, then `vvp sim`, open `gtkwave beacon.vcd`, and synthesize with Yosys.
8. Run the Blender script to render the final overview.

10. Conclusion

AeroSight shows that the full chain of a satellite project can be explored on an ordinary laptop. In one project, an AI model, a database, an orbit, a mechanical design, a circuit board, an antenna and a digital chip were built, checked and connected. The most important discovery

was that a satellite is not one large object but many small systems that have to talk to each other, and that each of them can be learned step by step.

This report is a record of a first step, and the limitations are stated openly. The next stage is to replace the simple parts with real data and real hardware, one improvement at a time.

"Dreams don't need a lab to start. This is my beginning."

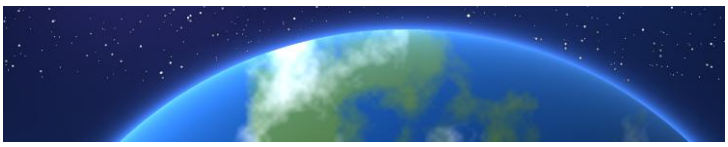


Figure 16. Earth's horizon: the view the satellite is designed to look at.

Tools and Credits

Python, NumPy, Pandas, Matplotlib, OpenCV, PyTorch, Ultralytics YOLOv8, Streamlit, SQLite, FreeCAD, KiCad, Icarus Verilog, Verilator, GTKWave, Yosys, Blender and Three.js. All of these are free or open-source tools, and the credit for the capability belongs to their developers and communities.